

Lab 2: Data Carving

What You Need for this lab

- Install Virtualbox : <https://www.virtualbox.org/wiki/Downloads>
- Install Kali 2021.4. : <https://old.kali.org/kali-images/kali-2021.4/>
 - Notes: Suggest You configure the disk size of Kali VM 80G because the size of each leakage cases image is 30G+
- Run a tool installation script instructions, or you can simply follow the commands below : (the script ONLY is tested on Kali 2021.4)
 - `wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Help/tool-install-zsh.sh`
 - `chmod +x tool-install-zsh.sh`
 - `./tool-install-zsh.sh`

Example scenarios

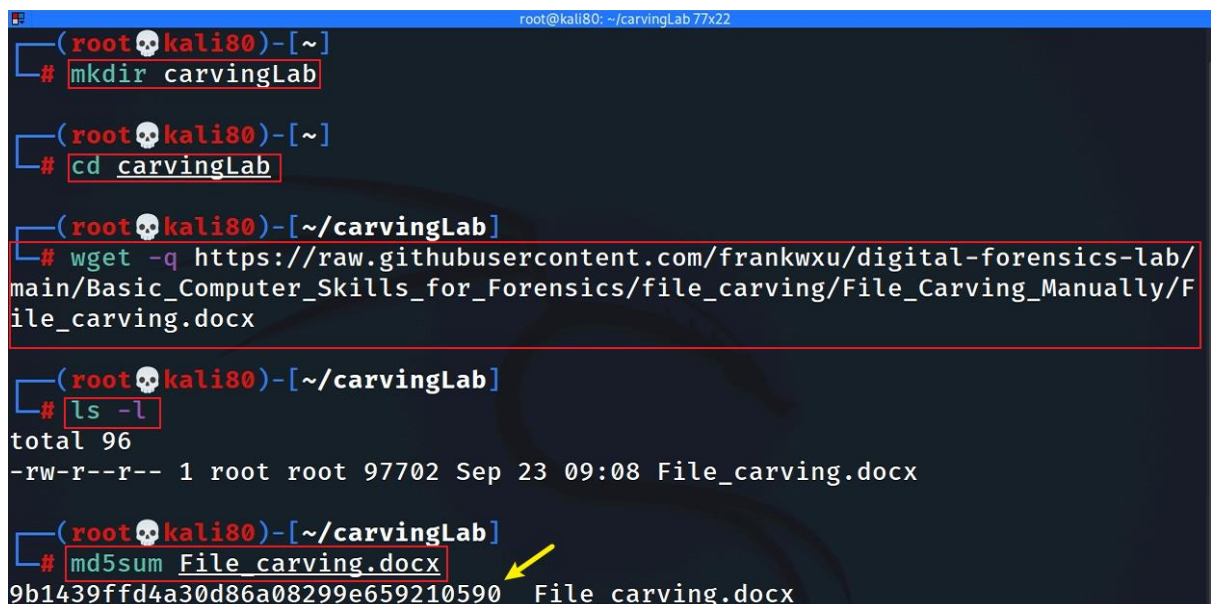
- Scenario 1: A file (A) is hidden inside of another file (B). You can't open the file B because the B's header is corrupted.
- Scenario 2: A suspect deleted files. The files contains an important information. A file occupies a few clusters. Unfortunately, some clusters are reused (overwritten) by new files.

A forensic expert really wants to recover files, even a partial files.

1. Extracting images from a corrupted Word document

Step 1.

- Prepare required files



```
root@kali80: ~/carvingLab 77x22
(root@kali80)-[~]
# mkdir carvingLab

(root@kali80)-[~]
# cd carvingLab

(root@kali80)-[~/carvingLab]
# wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/File_Carving_Manually/File_carving.docx

(root@kali80)-[~/carvingLab]
# ls -l
total 96
-rw-r--r-- 1 root root 97702 Sep 23 09:08 File_carving.docx

(root@kali80)-[~/carvingLab]
# md5sum File_carving.docx
9b1439ffd4a30d86a08299e659210590 File_carving.docx
```

A terminal window on a Kali Linux system. The user creates a directory named 'carvingLab', navigates into it, and uses 'wget' to download a file named 'File_carving.docx' from a GitHub repository. After the download, the user runs 'ls -l' to show the file's details. Finally, the user runs 'md5sum File_carving.docx' to generate a checksum for the file, which is displayed as '9b1439ffd4a30d86a08299e659210590 File_carving.docx'. A yellow arrow points to the md5sum command and its output.

Step 2.

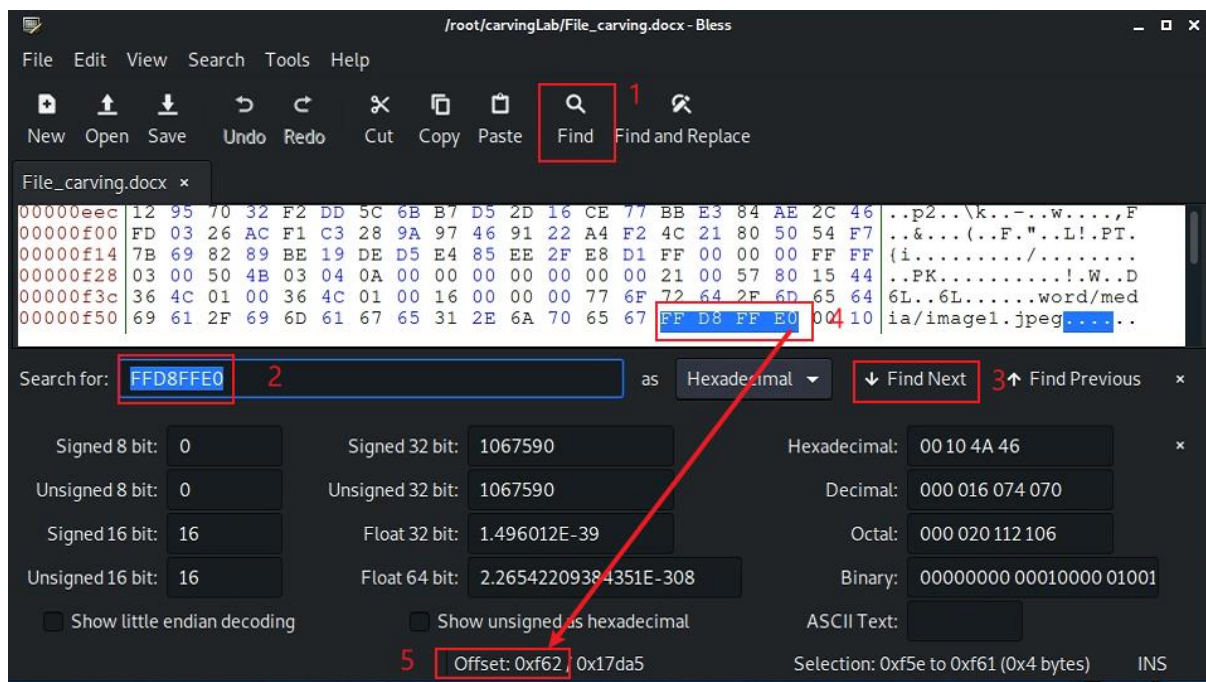
- View the file in a hex editor

```
(root@kali80)-[~/carvingLab]
# apt-get install bless
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bless is already the newest version (0.6.0-7).
0 upgraded, 0 newly installed, 0 to remove and 522 not upgraded.

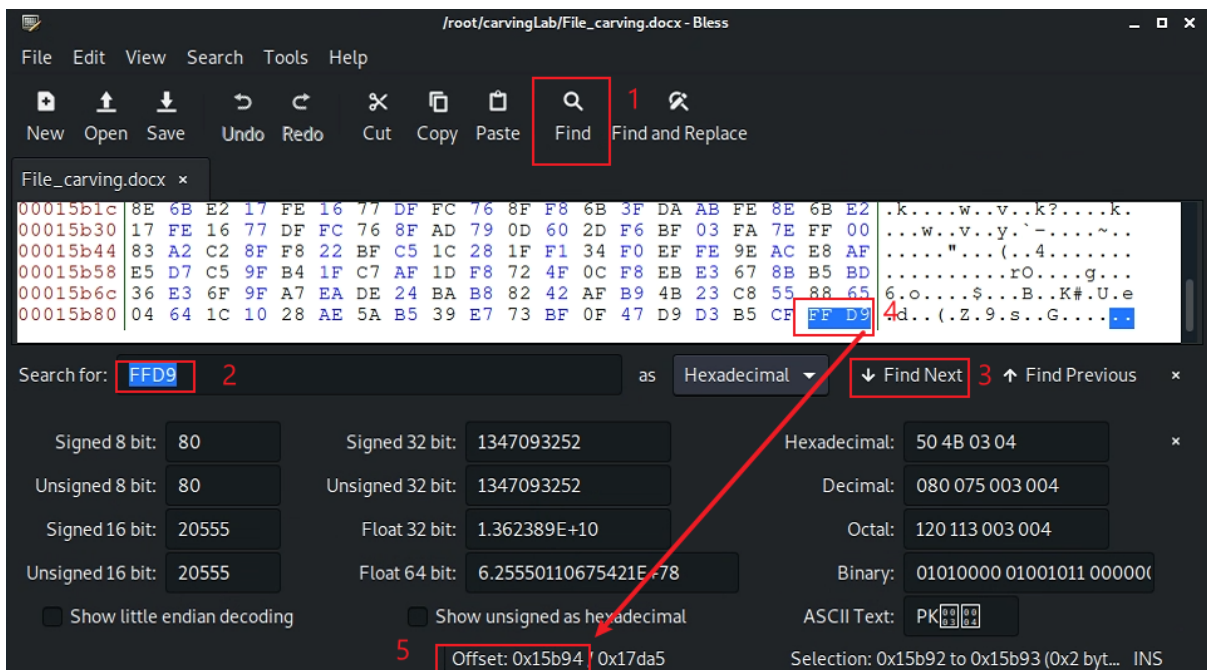
(root@kali80)-[~/carvingLab]
# bless File carving.docx
```

Step 3.

- Search file header start offset – 0F5E

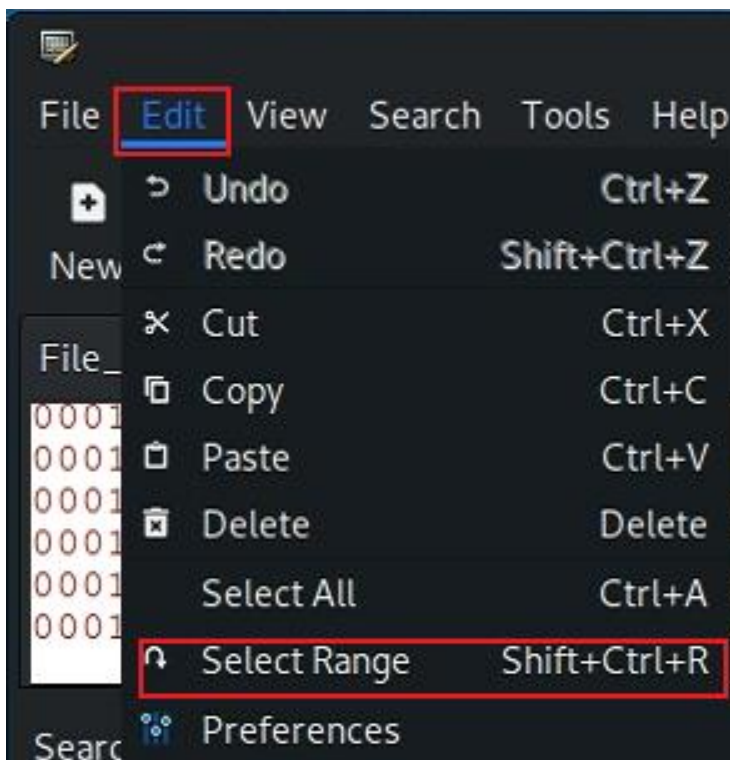


- Search file trailer ends offset – 15B93

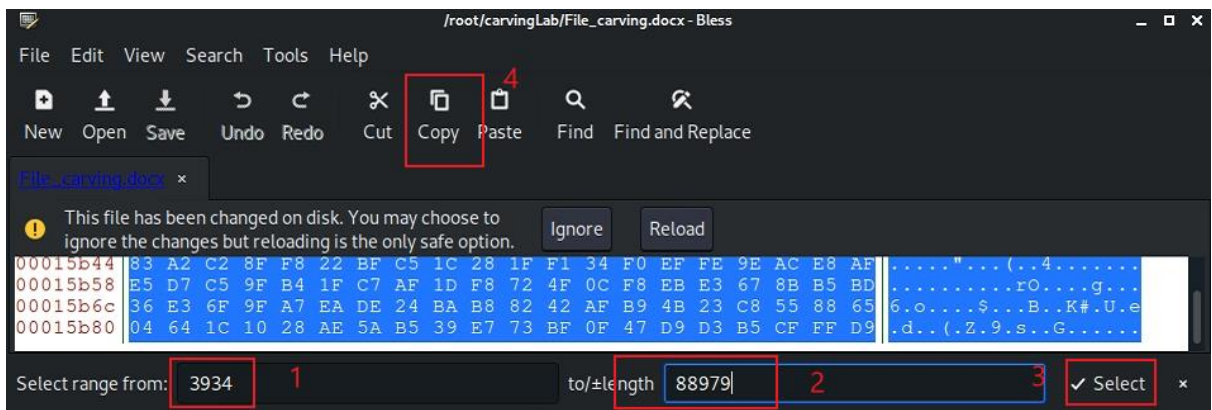


– Select hex from header to tail

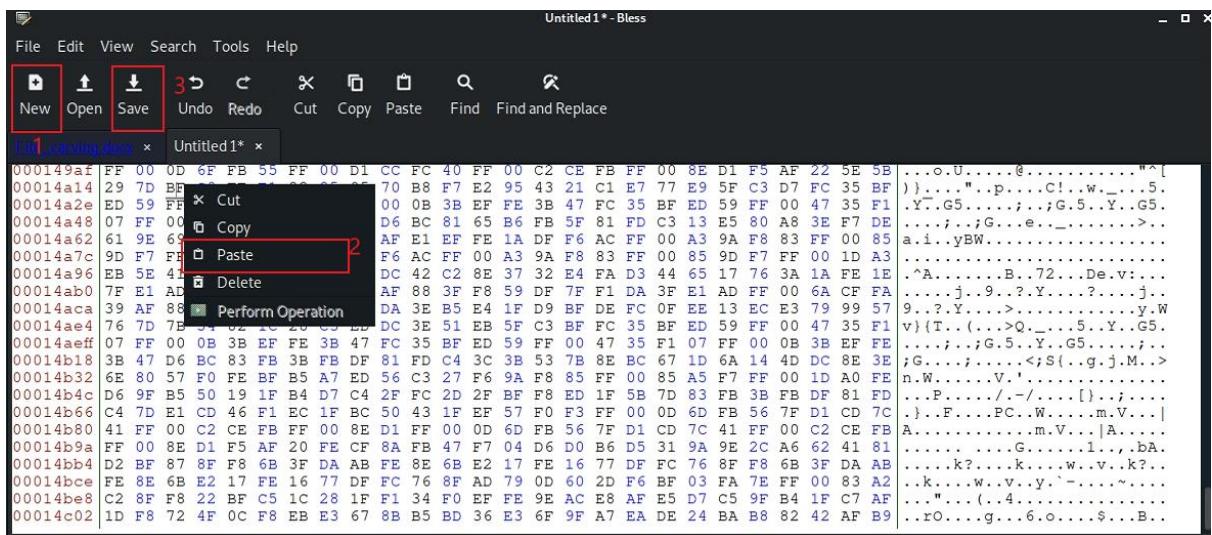
- $(0F5E)_{16} = (3934)_{10}$
- $(15B93)_{16} = (88979)_{10}$



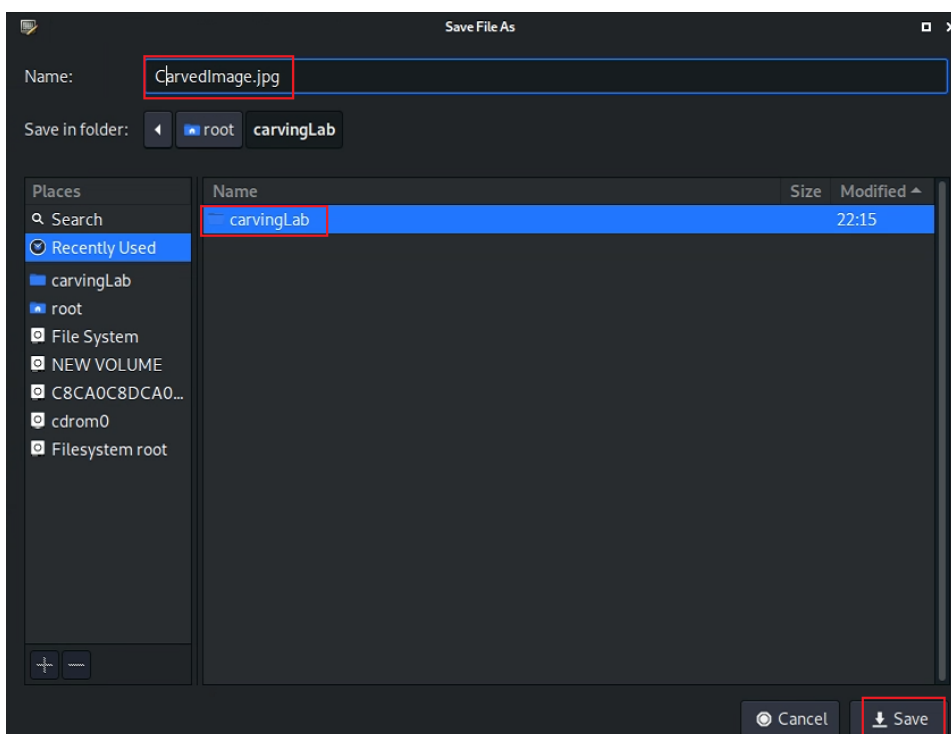
– Copy the selection



– Paste the selection

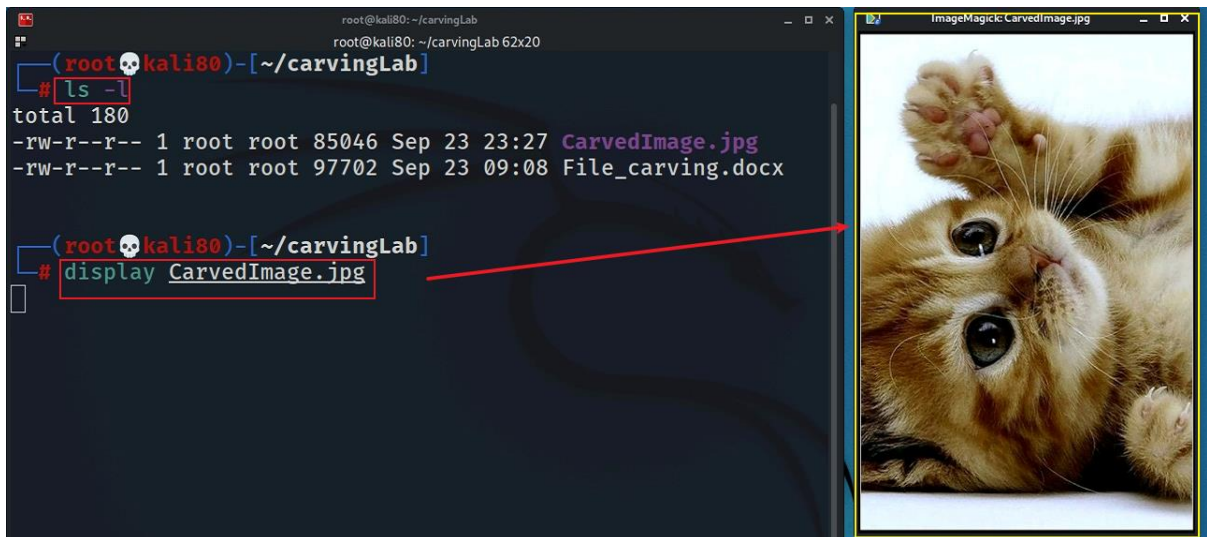


– Save the image



Step 4.

- Show the carved image



2. Carving/Recovering a USB image

- Prepare a USB image for file carving

Step 1.

- Download the zipped USB image



- Compute hashes



- List the content of the zipped file

```
(root@kali80) - [~/carvingLab]
# 7z l 120M.7z

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,2 CPUs Intel(R) Xe
on(R) Gold 6226R CPU @ 2.90GHz (50650),ASM,AES-NI)

Scanning the drive for archives:
1 file, 36720470 bytes (36 MiB)

Listing archive: 120M.7z

--
Path = 120M.7z
Type = 7z
Physical Size = 36720470
Headers Size = 214
Method = LZMA2:24
Solid = +
Blocks = 1

content of the zip
┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┐
│ Date      │ Time     │ Attr     │ Size      │ Compressed │ Name      │
├──────────┴──────────┴──────────┴──────────┴──────────┴──────────┤
│ 2021-09-22 22:59:31 D...A      │          0          │          0 │ 120M      │
│ 2021-09-22 22:59:31 ....A      │ 124780544          │ 36720256 │ 120M/usb_fat_carving.001 │
│ 2021-09-22 22:59:32 ....A      │ 1685              │          │ 120M/usb_fat_carving.001.txt │
├──────────┬──────────┬──────────┬──────────┬──────────┬──────────┤
│ 2021-09-22 22:59:32          │ 124782229          │ 36720256 │ 2 files, 1 folders │
└──────────┴──────────┴──────────┴──────────┴──────────┴──────────┘
```

- List the content of the zipped file

```
(root@kali80) - [~/carvingLab]
# 7z e 120M.7z

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,2 CPUs Intel(R) Xe
on(R) Gold 6226R CPU @ 2.90GHz (50650),ASM,AES-NI)

Scanning the drive for archives:
1 file, 36720470 bytes (36 MiB)

Extracting archive: 120M.7z

--
Path = 120M.7z
Type = 7z
Physical Size = 36720470
Headers Size = 214
Method = LZMA2:24
Solid = +
Blocks = 1

Everything is Ok

Folders: 1
Files: 2
Size: 124782229
Compressed: 36720470
```

- Verify the hashes

```

(root@kali80)-[~/carvingLab]
# hashdeep -c md5,sha1 usb fat carving.001
%%%% HASHDEEP-1.0
%%%% size,md5,sha1,filename
## Invoked from: /root/carvingLab
## # hashdeep -c md5,sha1 usb_fat_carving.001
##
124780544,ba4a1d0ba49f4a6667b00a3b3e85e604,bcc2d49fd49c9521ecb1739f6542c6bf327375ef,/root/carvingLab/usb_fat_carving.001

(root@kali80)-[~/carvingLab]
# cat usb fat carving.001.txt | grep "checksum"
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified

```

Step 2.

- Exam the content of the USB
- Display partitions

```

(root@kali80)-[~/carvingLab]
# fdisk -l usb fat carving.001
Disk usb_fat_carving.001: 119 MiB, 124780544 bytes, 243712 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa1159a00

Device                Boot Start    End Sectors  Size Id Type
usb_fat_carving.001p1 *      128 243711  243584 118.9M  e W95 FAT16 (LBA)

```

- Find deleted files


```

(root@kali80)-[~/carvingLab]
# fls -o 128 usb_fat_carving.001
r/r 3: USB (Volume Label Entry)
d/d 6: System Volume Information
r/r * 7: _est
r/r 10: .dropbox.device
d/d * 13: old File Carving_files
r/r * 15: B_ub_poe4.bmp
r/r * 18: B_zoom-eubie-mono.bmp
r/r * 21: Ballardlab8.java
r/r * 24: brittLab10.java
r/r * 27: DO_example.doc
r/r * 30: DO_example2.doc deleted
r/r * 33: G_BuiltForThis.gif
r/r * 35: G_zoom-sc.gif
r/r * 37: H_Form.html
r/r * 39: H_hello.html
r/r * 41: J_ub_law.jpg
r/r * 44: J_ub_night.jpg
r/r * 47: nps-2008-jean_outlook.pst
r/r * 50: P_CAS-zoom-6.png
r/r * 53: P_MSB_1_zoom.png
r/r * 57: pd_Evidence_search_techniques.pdf
r/r * 61: pd_Forensic_Report_Template.pdf
r/r * 64: pp_Number_Systems.pptx
r/r * 67: pp_one_page.pptx
r/r 69: readme.docx
r/r 70: readme.txt deleted
r/r * 71: _tf_1.rtf
r/r * 74: T_eubie-iphone-5-8.tiff
r/r * 78: T_youknowus-iphone-5-8.tiff
r/r * 81: W_axloadcomplete.wav
r/r * 84: W_speaker_test_sound.wav
r/r * 86: Z_file.zip
r/r * 88: 江莎莎行踪轨迹.doc
r/r * 91: nps-2008-jean_outlook.pst
v/v 3889667: $MBR
v/v 3889668: $FAT1
v/v 3889669: $FAT2
V/V 3889670: $OrphanFiles

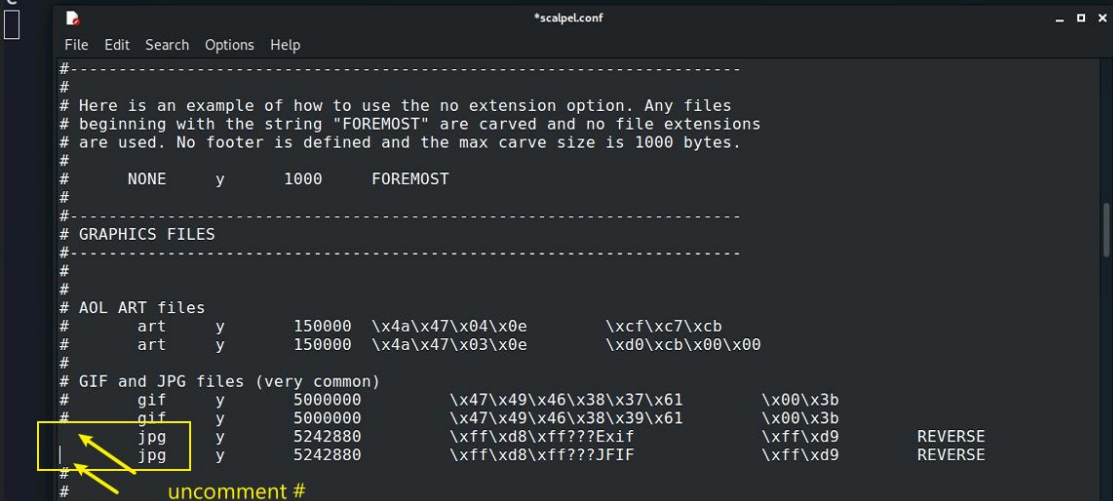
```

- Decide which file types need to carve

```

(root@kali80)-[~/carvingLab]
# leafpad /etc/scalpel/scalpel.conf
/usr/share/themes/Kali-Dark/gtk-2.0/gtkrc:39: Unable to find include file: "apps.rc"
/usr/share/themes/Kali-Dark/gtk-2.0/gtkrc:40: Unable to find include file: "hacks.rc"
/usr/share/themes/Kali-Dark/gtk-2.0/gtkrc:41: Unable to find include file: "hacks-dark.r
c"

```



```

#-----
#
# Here is an example of how to use the no extension option. Any files
# beginning with the string "FOREMOST" are carved and no file extensions
# are used. No footer is defined and the max carve size is 1000 bytes.
#
#      NONE      y      1000      FOREMOST
#
#-----
# GRAPHICS FILES
#-----
#
# AOL ART files
#      art      y      150000    \x4a\x47\x04\x0e    \xcf\x77\xcb
#      art      y      150000    \x4a\x47\x03\x0e    \xd0\xcb\x00\x00
#
# GIF and JPG files (very common)
#      gif      y      5000000    \x47\x49\x46\x38\x37\x61    \x00\x3b
#      gif      y      5000000    \x47\x49\x46\x38\x39\x61    \x00\x3b
#      jpg      y      5242880    \xff\xd8\xff???Exif        \xff\xd9    REVERSE
#      jpg      y      5242880    \xff\xd8\xff???JFIF        \xff\xd9    REVERSE
#
#-----
# uncomment #
# PNG

```

- Save it and quit!
- Show help


```
(root@kali80)-[~/carvingLab]
# scalpel -h
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.
Carves files from a disk image based on file headers and footers.

Usage: scalpel [-b] [-c <config file>] [-d] [-h|V] [-i <file>]
      [-m blocksize] [-n] [-o <outputdir>] [-O num] [-q clusterize]
      [-r] [-s num] [-t <blockmap file>] [-u] [-v]
      <imgfile> [<imgfile>] ...

-b Carve files even if defined footers aren't discovered within
  maximum carve size for file type [foremost 0.69 compat mode].
-c Choose configuration file.
-d Generate header/footer database; will bypass certain optimizations
  and discover all footers, so performance suffers. Doesn't affect
  the set of files carved. **EXPERIMENTAL**
-h Print this help message and exit.
-i Read names of disk images from specified file.
-m Generate/update carve coverage blockmap file. The first 32bit
  unsigned int in the file identifies the block size. Thereafter
  each 32bit unsigned int entry in the blockmap file corresponds
  to one block in the image file. Each entry counts how many
  carved files contain this block. Requires more memory and
  disk. **EXPERIMENTAL**
-n Don't add extensions to extracted files.
-o Set output directory for carved files.
-O Don't organize carved files by type. Default is to organize carved files
  into subdirectories.
-p Perform image file preview; audit log indicates which files
  would have been carved, but no files are actually carved.
-q Carve only when header is cluster-aligned.
-r Find only first of overlapping headers/footers [foremost 0.69 compat mode].
-s Skip n bytes in each disk image before carving.
-t Set directory for coverage blockmap. **EXPERIMENTAL**
-u Use carve coverage blockmap when carving. Carve only sections
  of the image whose entries in the blockmap are 0. These areas
  are treated as contiguous regions. **EXPERIMENTAL**
```

Step 3.

- Carving the USB image

```
(root@kali80)-[~/carvingLab]
# scalpel usb_fat_carving.001 -o output
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/root/carvingLab/usb_fat_carving.001"

Image file pass 1/2.
usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" --> 8 f
iles
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" --> 9 f
iles
Carving files from image.
Image file pass 2/2.
usb_fat_carving.001: 100.0% |*****| 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 0 seconds.
```

- Show carved files

```
(rootkali80)-[~/carvingLab]
# tree output
output
├── audit.txt
├── jpg-0-0
│   ├── 00000000.jpg
│   ├── 00000001.jpg
│   ├── 00000002.jpg
│   ├── 00000003.jpg
│   ├── 00000004.jpg
│   ├── 00000005.jpg
│   ├── 00000006.jpg
│   └── 00000007.jpg
├── jpg-1-0
│   ├── 00000008.jpg
│   ├── 00000009.jpg
│   ├── 00000010.jpg
│   ├── 00000011.jpg
│   ├── 00000012.jpg
│   ├── 00000013.jpg
│   ├── 00000014.jpg
│   ├── 00000015.jpg
│   └── 00000016.jpg
└── 2 directories, 18 files
```

carved files

- Show audit log

```
(rootkali80)-[~/carvingLab]
# cat output/audit.txt

Scalpel version 1.60 audit file
Started at Fri Sep 24 11:20:43 2021
Command line:
scalpel usb_fat_carving.001 -o output

Output directory: /root/carvingLab/output
Configuration file: /etc/scalpel/scalpel.conf

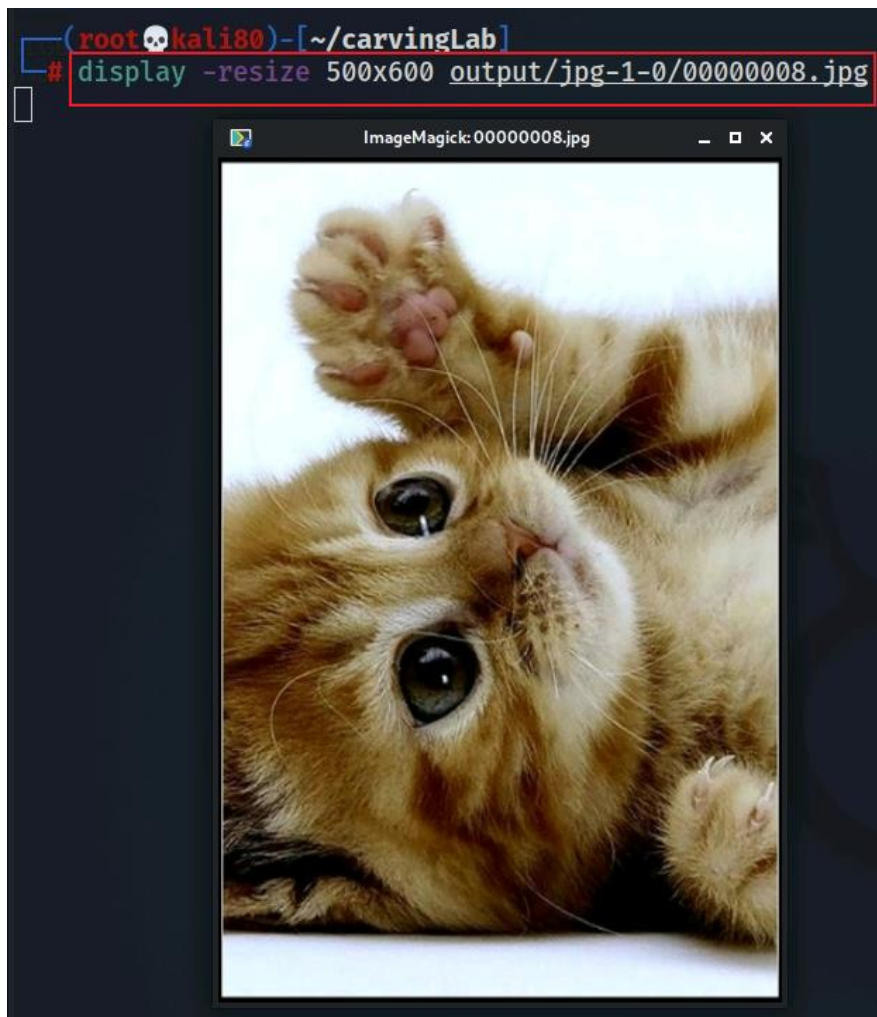
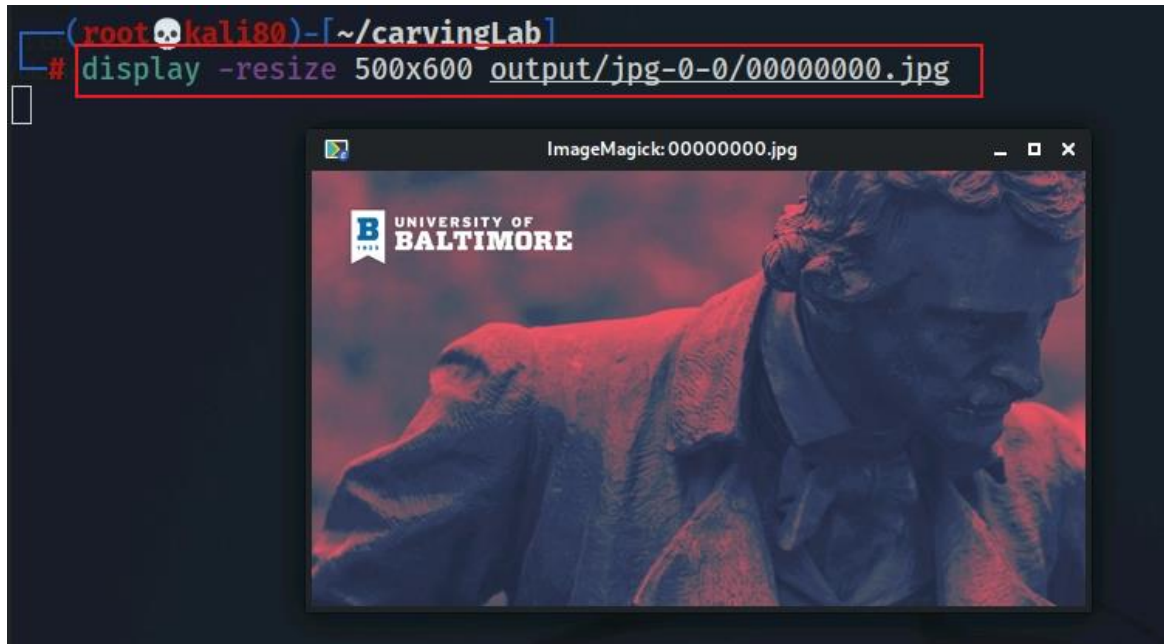
Opening target "H="

The following files were carved:
File                Start           Chop           Length           Extracted From
00000010.jpg        3796992         NO             3148500          usb_fat_carving.001
00000009.jpg        425822          NO             4875802          usb_fat_carving.001
00000008.jpg        335872          NO             4965752          usb_fat_carving.001
00000002.jpg        6938624         NO             6868             usb_fat_carving.001
00000001.jpg        5285888         NO             1659604          usb_fat_carving.001
00000000.jpg        3897344         NO             3048148          usb_fat_carving.001
00000004.jpg        15427584        NO             4223822          usb_fat_carving.001
00000003.jpg        12881920        NO             4256310          usb_fat_carving.001
00000006.jpg        19638272        NO             5229050          usb_fat_carving.001
00000005.jpg        17121280        NO             4248530          usb_fat_carving.001
00000007.jpg        21358592        NO             5183267          usb_fat_carving.001
00000016.jpg        36671488        NO             2889262          usb_fat_carving.001
00000015.jpg        36571136        NO             2989614          usb_fat_carving.001
00000014.jpg        36393610        NO             3167140          usb_fat_carving.001
00000013.jpg        36352448        NO             3208302          usb_fat_carving.001
00000012.jpg        35590996        NO             3969754          usb_fat_carving.001
00000011.jpg        35350242        NO             4210508          usb_fat_carving.001

Completed at Fri Sep 24 11:20:43 2021
```


Step 4.

- Display two carved jpg image



YOU MUST SUBMIT A FULL-SCREEN IMAGE FOR FULL CREDIT!

Save the document with the filename "**YOUR NAME Lab 2.pdf**", replacing "YOUR NAME" with your real name.

Email the image to the instructor as an attachment to an e-mail message. Send it to: **xxx@fe.edu.vn** with a subject line of "**Lab 2 From YOUR NAME**", replacing "YOUR NAME" with your real name.